T.C.

MARMARA UNIVERSITY

FACULTY of ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

CSE4297 Engineering Project I Proposal

Title of the Project

Face Recognition and Identity Verification using Cloud-Based SaaS Models

Group Members

150121025 Ahmet Abdullah Gültekin

150120005 Ayşe Gülsüm Eren

150123825 Ayşenur Arıcı

Supervised by

Assoc. Dr. Mustafa Ağaoğlu

1. Aim of the Project

The proliferation of cloud-native applications and the increasing demand for scalable, resilient software systems have made microservices architecture a fundamental paradigm in modern software engineering. Traditional monolithic applications face significant challenges in terms of scalability, maintainability, and deployment flexibility, particularly when dealing with high-throughput real-time data processing and multi-modal authentication requirements.

This project aims to design and implement a comprehensive microservices-based system that addresses critical challenges in distributed computing, focusing on real-time data processing, authentication services, and cross-platform integration. The system will demonstrate advanced software engineering principles including hexagonal architecture, SOLID principles, design patterns, and professional development practices.

The motivation for this study stems from the growing need for educational institutions and enterprises to adopt scalable, secure, and maintainable software architectures. Current systems often struggle with bottlenecks in authentication processes, limited scalability during peak usage, and inadequate real-time data processing capabilities. Our project will create a foundational microservices platform that can be extended for various applications including educational technology, enterprise software, and real-time monitoring systems.

The primary goal is to develop a production-ready microservices ecosystem that showcases modern software engineering practices while solving real-world problems in distributed system design.

2. Methodology

We will follow a clear and systematic process, combining theoretical knowledge with practical implementation.

Architectural Approach

- Each microservice will use a hexagonal architecture for loose coupling between business logic and external systems.
- We will use an API Gateway for orchestration and circuit breaker patterns for fault tolerance.

- Event-driven architecture will be implemented with message queues and streaming platforms for asynchronous communication.

Development Methodology

- We will follow Agile development with two-week sprints.
- We will use Test-Driven Development (TDD) and CI/CD pipelines.
- All services will be containerized with Docker and deployed using Kubernetes.

Service Design Strategy

Core services will include:

1. Authentication Service – with multi-modal verification.
2. Data Processing Service – for real-time analytics.
3. Communication Service – with WebSocket and REST APIs.
4. Monitoring Service – for system health and performance metrics.

All services will follow SOLID principles with dependency injection and strong error handling.

Technology Integration

- Backend: Node.js (TypeScript) and Python for data processing.
- Databases: PostgreSQL for relational data, MongoDB for documents, Redis for caching.
- Messaging: Apache Kafka for event streaming, RabbitMQ for task queues.
- Frontend: React (web), Flutter (mobile), Electron (desktop).
- DevOps: Docker, Kubernetes, Prometheus, Grafana.

Quality Assurance

- Unit test coverage >90%.
- Integration, performance, and security testing.
- Documentation with OpenAPI.
- Automated linting, code reviews, and static analysis.

3. Software/Hardware Requirements

Development Environment

- OS: Windows 10/11, macOS, Linux
- IDE: Visual Studio Code, Jetbrains
- Version Control: GitHub

Backend

- Language: Java, SpringBoot
- Database: PostgreSQL

Frontend

- Web: React
- Mobile: Flutter
- Desktop: Java

Tools

- API: Swagger
- Testing: Postman

4. Draft Time Plan

First Semester – Design & Architecture

Weeks 1–2: Requirements Analysis & System Design

Weeks 3-4: Microservices Architecture Design

Weeks 4-5: Environment Setup & Prototyping

Weeks 5-6: Detailed Service Design & Documentation

Weeks 7: Deliverables & Presentation

Second Semester – Implementation & Integration

Weeks 1–3: Core Services Development

Weeks 4–6: Communication & Integration

Weeks 7–9: Frontend Development

Weeks 10–12: System Integration & Testing

Weeks 13–14: Deployment & Final Presentation

5.References

[1] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.

[2] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[3] Vernon, V. (2016). *Domain-Driven Design Distilled*. Addison-Wesley Professional.

[4] Fowler, M. (2014). "Microservices: A Definition of This New Architectural Term." Martin Fowler's Blog. Retrieved from https://martinfowler.com/articles/microservices.html